

Libro: Python Data Science Handbook -Jake VanderPlas

Imputación de Datos Faltantes

Uno de los aspectos más comunes en la ingeniería de características (feature engineering) es el manejo de datos faltantes. En el capítulo 16 se discutió cómo los objetos DataFrame suelen usar NaN para marcar valores ausentes. Por ejemplo, podríamos tener un conjunto de datos como el siguiente:

```
import numpy as np

X = np.array([[np.nan, 0, 3],
              [3, 7, 9],
              [3, 5, 2],
              [4, np.nan, 6],
              [8, 8, 1]])

y = np.array([14, 16, -1, 8, -5])
```

Cuando aplicamos un modelo de machine learning típico a estos datos, es necesario reemplazar primero los valores faltantes con algún valor adecuado. A este proceso se le conoce como **imputación** de valores faltantes. Las estrategias van desde las más simples (por ejemplo, reemplazar con la media de la columna) hasta las más sofisticadas (como modelos iterativos, KNN o incluso bosques aleatorios).

Las aproximaciones más avanzadas suelen ser muy específicas de cada aplicación, pero a continuación presentamos desde las básicas hasta las más potentes disponibles en la actualidad (2025-2026).

1. Imputación básica con SimpleImputer (scikit-learn)

```
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy='mean')          # Otras opciones: 'median',
'most_frequent', 'constant'
X2 = imp.fit_transform(X)

print(X2)
```

Resultado aproximado:

```
text
[[4.5 0.  3. ]
 [3.  7.  9. ]
 [3.  5.  2. ]
 [4.  5.  6. ]    ← valor imputado con la media de la columna
 [8.  8.  1. ]]
```

Este dato imputado ya se puede usar directamente en un modelo, por ejemplo:

Python

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression().fit(X2, y)
```

```
print(model.predict(X2))
```

2. Técnicas avanzadas de imputación (recomendadas en la práctica actual)

Método	Clase / Librería	Ventajas	Desventajas	Cuándo usarlo
SimpleImputer	sklearn.impute.SimpleImputer	Muy rápido, simple, interpretable	Ignora relaciones entre variables	Datasets pequeños / baseline
IterativeImputer	sklearn.impute.IterativeImputer	Modela relaciones entre variables, muy preciso	Más lento, requiere varias iteraciones	Datasets medianos/grandes, relaciones lineales
KNNImputer	sklearn.impute.KNNImputer	No paramétrico, captura relaciones no lineales	Sensible a escala, lento en datasets grandes	Datos con estructura de clústeres
MissForest (equivalente)	IterativeImputer + ExtraTreesRegressor	Muy preciso, maneja relaciones no lineales complejas	Más lento que IterativeImputer	Alta precisión necesaria
MICE (Multiple Imputation)	sklearn + fancyimpute / miceforest	Considera incertidumbre, ideal para inferencia estadística	Complejo de implementar e interpretar	Análisis estadístico formal

Ejemplo práctico: IterativeImputer (muy recomendado)

```
from sklearn.experimental import enable_iterative_imputer # Necesario en versiones antiguas
from sklearn.impute import IterativeImputer
from sklearn.ensemble import ExtraTreesRegressor           # o BayesianRidge, etc.
```

```
# Configuración recomendada (equivalente aproximado a MissForest)
imp_iter = IterativeImputer(
    estimator=ExtraTreesRegressor(n_estimators=100, random_state=42),
    max_iter=10,
    random_state=42,
    verbose=1
)

X_iter = imp_iter.fit_transform(X)
print("Datos imputados con IterativeImputer:\n", np.round(X_iter, 2))
```

Ventajas principales:

- Itera varias veces mejorando la imputación
- Usa un modelo predictivo por columna (puede ser muy potente con árboles)

Ejemplo: KNNImputer

```
from sklearn.impute import KNNImputer

imp_knn = KNNImputer(n_neighbors=3, weights='distance')
X_knn = imp_knn.fit_transform(X)
print("Datos imputados con KNN:\n", np.round(X_knn, 2))
```

Ideal cuando los datos tienen patrones de similitud entre observaciones.

3. Pipelines de características (Feature Pipelines)

Hacer las transformaciones a mano puede volverse tedioso, especialmente cuando queremos combinar varios pasos. Por ejemplo, un pipeline típico podría ser:

1. Imputar valores faltantes (con una técnica avanzada)
2. Generar características polinómicas
3. Ajustar un modelo de regresión

Scikit-learn ofrece la clase Pipeline (o la función `make_pipeline`) para encadenar estos pasos de forma elegante:

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
```

```

# Pipeline con imputación avanzada
pipeline = make_pipeline(
    IterativeImputer(estimator=ExtraTreesRegressor(n_estimators=50), random_state=42),
    PolynomialFeatures(degree=2),
    LinearRegression()
)

# Entrenamos directamente con datos con valores faltantes
pipeline.fit(X, y)

# Predicciones
print("Predicciones del pipeline:", pipeline.predict(X))
print("Valores reales: ", y)

```

Resultado esperado (ejemplo): Las predicciones suelen ser muy cercanas a los valores reales cuando el modelo es adecuado y no hay sobreajuste grave (en este caso pequeño dataset de ejemplo tiende a sobreajustar).

Nota importante: En la práctica real nunca evalúes el modelo en los mismos datos con los que lo entrenaste. Usa validación cruzada (`cross_val_score`) para obtener una estimación realista del rendimiento.

Recomendaciones finales (2026)

- **Baseline rápido** → `SimpleImputer(strategy='median')` (robusto a outliers)
- **Mejor precisión general** → `IterativeImputer` con `ExtraTreesRegressor` o `RandomForestRegressor` (equivalente moderno a `MissForest`)
- **Datos con estructura espacial/local** → `KNNImputer`
- **Datasets muy grandes** → Considera aproximaciones más rápidas como `SimpleImputer` + corrección posterior o muestreo
- **Análisis estadístico serio** → Múltiples imputaciones (paquetes como `miceforest`)